Guion de la presentación DomoVida · versión para la comisión (corregida tras el ensayo del 11/11)

Correcciones del profesor en el ensayo: objetivo con simulador IoT, cronograma con semanas 16 a 18 y menos tiempo (hablaste 9 min). Meta: 10 minutos de exposición con el video incluido. Menos es más: sin opiniones personales; demuestra competencias. Usa puntero y no te pongas delante de la pantalla.

Los 8 números que debes saber:

54 milisegundos (servidor en Santiago) · más de 1,5 segundos antes (EE. UU. y Brasil)

7 cuidadores · SUS 92,1 de 100 (meta 70)

20 usuarios a la vez, 0 errores · 30 de 30 avisos al celular

24 historias de usuario, 21 terminadas · 189 commits

1. Portada

[20 s] Buenas tardes, profesor Cristian, compañeros. Mi nombre es Andrés Vergara Acevedo. Mi proyecto se llama DomoVida y cuida a personas mayores que viven solas. Funciona sin cámaras y avisa al cuidador en el momento de la caída.

2. Integrantes

[15 s] Este es un proyecto individual: asumí tres roles, Product Owner, Scrum Master y desarrollador. Mi formación como tecnólogo en informática biomédica me ayudó a entender el área de la salud.

3. Descripción del proyecto

[25 s] Chile envejece rápido: casi un tercio de la población tendrá 60 años o más en 2050, y uno de cada tres mayores de 65 se cae al año. Quien vive solo puede pasar horas en el suelo. Las cámaras podrían ayudar, pero a nadie le gustaría que grabaran a su mamá bañándose o vistiéndose. DomoVida no graba: funciona solo con sensores, avisa en segundos y sigue registrando aunque se corte internet.

Ojo: no digas «como todos sabemos»; parte directo con el dato.

4. ¿Cómo funciona DomoVida?

[20 s] ¿Cómo funciona DomoVida? Primero, el simulador IoT envía la caída con los mismos datos que enviaría un sensor real. Segundo, el servidor aplica la regla de alerta. Tercero, el aviso llega al panel y al celular: desde la caída simulada hasta el panel toma solo 54 milisegundos. Cuarto, el cuidador la atiende.

Ojo: no digas «el sensor detecta»; di «el simulador IoT envía la caída». Los 54 ms son desde la caída simulada hasta que la alerta aparece en el panel (no incluyen al cuidador).

5. Objetivos

[20 s] El objetivo general fue desarrollar un sistema de monitoreo y alerta, y para probarlo construí un simulador de sensores IoT que reemplaza al dispositivo físico. Lo dividí en tres objetivos medibles y los tres se cumplieron: el simulador y el modo sin internet; el servidor con los datos protegidos; y el panel del cuidador con los avisos.

Ojo (comentario del profesor): no construiste el dispositivo IoT; construiste el sistema de alertas y un simulador IoT para probarlo. Dilo así.

6. Alcances y limitaciones

[20 s] El prototipo incluye todo el software funcionando, con sensores simulados. Lo que queda para después es el hardware real, la inteligencia artificial y un piloto en hogares. Y lo digo claro: no reemplaza la atención médica.

7. Metodología

[20 s] Trabajé con Scrum adaptado a una sola persona: sprints de tres semanas, cada uno con planificar, construir, probar y mejorar. Todo está en GitHub: el tablero Kanban, el backlog, las fichas de sprint y un registro de pruebas con evidencias.

8. Sprints

[15 s] El desarrollo dura 15 semanas: cuatro sprints completos y un sprint de cierre con la Nota 3. Las semanas 16 a 18 son la Fase 3, la defensa ante la comisión. El avance fue más rápido de lo previsto: las pruebas de carga y con usuarios se adelantaron al Sprint 4.

Ojo: 15 semanas de desarrollo; la defensa es la Fase 3, semanas 16 a 18.

9. Tablero Kanban

[15 s] El trabajo se gestionó en un tablero Kanban público en GitHub: 24 historias de usuario, 21 terminadas, y 189 commits que dejan todo trazable.

Pronuncia «Kan-ban» (Canva es otra aplicación).

10. Cronograma

[15 s] Este es el cronograma: 15 semanas de desarrollo y, de la 16 a la 18, la Fase 3 con la defensa ante la comisión. Lo claro es lo planificado y lo oscuro lo realizado; varias etapas se adelantaron. Hoy estamos en el sprint de cierre.

Truco: oscuro = ya pinté = hecho.

11. Arquitectura

[25 s] Los sensores del hogar envían sus datos con una clave. El servidor está en Oracle Cloud, región Santiago. Caddy da la conexión segura, y la API FastAPI con su base SQLite están en la misma máquina: por eso la alerta llega en 54 milisegundos. (pausa) De ahí sale al panel en vivo y al celular. Render y Supabase guardan una copia en la nube.

Si preguntan por qué Oracle: está en Santiago, cerca de los usuarios, y es gratuito para siempre (Always Free).

12. Modelo de datos

[20 s] El modelo tiene tres tablas: sensores, eventos y alertas. Cada evento lleva un identificador único para que la sincronización no duplique los datos, y la tabla de alertas registra quién atendió y cuándo. La misma estructura funciona en SQLite y en PostgreSQL, y se expone en el estándar de salud HL7 FHIR.

Pronuncia «Póstgres» y «ache ele siete fáier».

13. Tecnologías

[20 s] Usé tecnologías abiertas. Python y FastAPI en el servidor. SQLite y PostgreSQL para los datos. React con TypeScript en el panel. Oracle Cloud, Docker y Caddy en la infraestructura. Para integrarse con salud, HL7 FHIR. Y la calidad se probó con Locust.

Haz una pausa corta después de cada grupo. Pronuncia «Lócust».

14. Seguridad y privacidad

[20 s] La privacidad está desde el diseño. No hay cámaras ni micrófonos. Solo los sensores con clave pueden enviar datos, y consultar los datos o atender alertas exige la sesión del cuidador. Los datos se seudonimizan, y la persona acepta un consentimiento informado, siguiendo los principios de la Ley 21.719.

15. Demostración

[60 s] Video de la demostración (clic en el video para reproducir, 27 s). Narra mientras corre: «Aquí el panel está en verde, con 8 de 8 sensores.» «Se genera una caída y aparece en rojo al instante.» «El aviso llega al celular del cuidador, como verán en la siguiente lámina.» «La marco como atendida y queda registrado quién y a qué hora.»

Ojo: en el ensayo no se muestra el sistema corriendo; el video está dentro de esta lámina. Para la comisión (demo en vivo) se usan los mismos 4 pasos.

16. Panel del cuidador

[15 s] Así se ve el panel: en verde cuando todo está bien, y en rojo, al instante, cuando hay una caída. El cuidador la atiende con un clic.

17. Aviso al celular y semáforo

[15 s] El aviso llega al celular del cuidador: en la prueba llegaron los 30 de 30. Y si un sensor deja de enviar datos, el resumen se pone amarillo e indica cuál y desde qué hora. Así, el sistema avisa solo cuando algo requiere atención.

18. Resultados

[25 s] Los resultados se midieron. Con el servidor en Santiago, la alerta llega en 54 milisegundos; con los servicios en Estados Unidos y Brasil tardaba más de un segundo y medio. Con 20 usuarios a la vez no hubo errores. Llegaron los 30 avisos al celular. (pausa) Y sin internet, las caídas se sincronizaron después, sin duplicados.

19. Validación SUS

[20 s] Siete cuidadores reales probaron el sistema. El puntaje de usabilidad fue noventa y dos coma uno de 100, un nivel excelente, sobre la meta de 70. Un participante valoró justamente que no se grabe imagen ni sonido de su madre.

20. Obstáculos

[20 s] Hubo obstáculos y cada uno dejó una mejora. La alerta era lenta, y la llevé a Santiago. Encontré brechas de seguridad, y las cerré. Corregí los datos inconsistentes con migraciones. Los cuidadores detectaron problemas que se convirtieron en mejoras. Y organicé mi trabajo en bloques según mis turnos.

21. Preguntas

[5 s] Muchas gracias. Quedo atento a sus preguntas.